LAB Experiment - 1

Problem Statement: Write a python program for Examination Result and Grace-Mark Validation that processes marks across multiple subjects, applies subject-wise pass criteria and a grace-mark limit, and generates final result status.

Aim: Python program to check pass status and apply grace marks without exceeding maximum marks or grace limit.

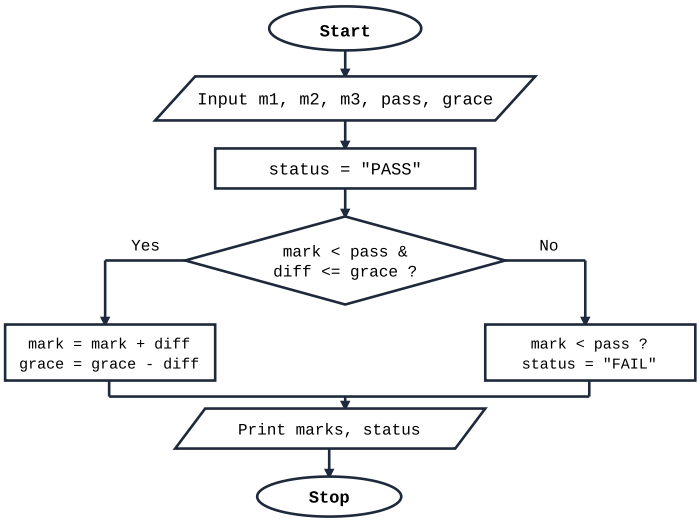
Analyze the problem (Identify the input, process, output):

- **Input:** marks of three subjects (m1, m2, m3), pass mark (pass_mark), and available grace marks (grace)
- **Process:** if mark < pass_mark and deficit <= grace, allocate needed marks from grace; else flag status as FAIL
- **Output:** updated marks and final result (PASS / FAIL)

Algorithm:

1. Start
2. Input m1, m2, m3, pass_mark, grace
3. Set status = "PASS"
4. For each mark, if mark < pass_mark: check if deficit <= grace; if yes, add deficit to mark and reduce grace; else set status = "FAIL"
5. Print updated marks and status
6. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT m1 INPUT m2 INPUT m3 INPUT pass_mark INPUT grace SET status = "PASS" IF m1 < pass_mark THEN SET diff = pass_mark - m1 IF diff <= grace THEN COMPUTE m1 = m1 + diff COMPUTE grace = grace - diff ELSE SET status = "FAIL" IF m2 < pass_mark THEN SET diff = pass_mark - m2 IF diff <= grace THEN COMPUTE m2 = m2 + diff COMPUTE grace = grace - diff ELSE SET status = "FAIL" IF m3 < pass_mark THEN SET diff = pass_mark - m3 IF diff <= grace THEN COMPUTE m3 = m3 + diff COMPUTE grace = grace - diff ELSE SET status = "FAIL" PRINT m1, m2, m3 PRINT status END
```

Python Program:

```
m1 = int(input("Enter marks 1: ")) m2 = int(input("Enter marks 2: ")) m3 =
int(input("Enter marks 3: ")) pass_mark = int(input("Enter pass mark: ")) grace =
int(input("Enter grace marks: ")) status = "PASS" if m1 < pass_mark: diff = pass_mark - m1
if diff <= grace: m1 = m1 + diff grace = grace - diff else: status = "FAIL" if m2 <
pass_mark: diff = pass_mark - m2 if diff <= grace: m2 = m2 + diff grace = grace - diff else:
status = "FAIL" if m3 < pass_mark: diff = pass_mark - m3 if diff <= grace: m3 = m3 + diff
grace = grace - diff else: status = "FAIL" print("Marks:", m1, m2, m3) print("Result:",
status)
```

Test Cases:

- **Case-1:**
Input: m1=33, m2=40, m3=50, pass=35, grace=5
Expected: Marks: 35 40 50, Result: PASS
Actual: Marks: 35 40 50, Result: PASS

- **Case-2:**
Input: m1=25, m2=60, m3=70, pass=35, grace=5
Expected: Marks: 25 60 70, Result: FAIL
Actual: Marks: 25 60 70, Result: FAIL

LAB Experiment - 2

Problem Statement: Write a python program for Attendance Recovery Computation that calculates current attendance percentage and determines the minimum consecutive classes required to reach an eligibility threshold.

Aim: Python program to compute remaining consecutive classes to achieve minimum required attendance percentage.

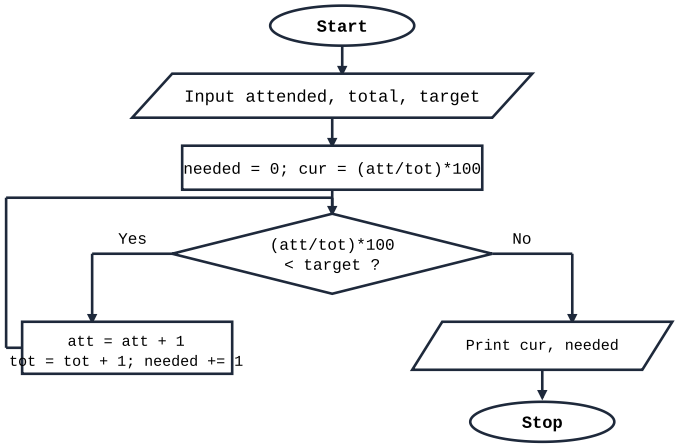
Analyze the problem (Identify the input, process, output):

- **Input:** attended classes (attended), total conducted classes (total_classes), target percentage (target)
- **Process:** compute current percentage; increment attended and total by 1 in a loop until percentage >= target
- **Output:** current percentage and minimum consecutive classes needed

Algorithm:

1. Start
2. Input attended, total_classes, target
3. Compute current_pct = (attended / total_classes) * 100
4. Set needed = 0, temp_att = attended, temp_tot = total_classes
5. While (temp_att / temp_tot) * 100 < target: temp_att = temp_att + 1, temp_tot = temp_tot + 1, needed = needed + 1
6. Print current_pct and needed
7. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT attended INPUT total_classes INPUT target COMPUTE current_pct = (attended / total_classes) * 100 SET needed = 0 SET temp_att = attended SET temp_tot = total_classes WHILE (temp_att / temp_tot) * 100 < target DO COMPUTE temp_att = temp_att + 1 COMPUTE temp_tot = temp_tot + 1 COMPUTE needed = needed + 1 PRINT current_pct PRINT needed END
```

Python Program:

```
attended = int(input("Enter attended classes: ")) total_classes = int(input("Enter total classes: ")) target = float(input("Enter target percentage: ")) current_pct = (attended / total_classes) * 100 needed = 0 temp_att = attended temp_tot = total_classes while (temp_att / temp_tot) * 100 < target: temp_att = temp_att + 1 temp_tot = temp_tot + 1 needed = needed + 1 print("Current Attendance:", current_pct) print("Classes needed to reach target:", needed)
```

Test Cases:

- **Case-1:**
Input: attended=28, total=40, target=75.0
Expected: Current: 70.0, Needed: 8
Actual: Current: 70.0, Needed: 8

- **Case-2:**
Input: attended=30, total=40, target=75.0
Expected: Current: 75.0, Needed: 0
Actual: Current: 75.0, Needed: 0

LAB Experiment - 3

Problem Statement: Write a python program for ATM Transaction Sequence Simulation that processes deposits and withdrawals, maintains a simulated balance, enforces withdrawal limits, rejects invalid transactions, and produces a summary.

Aim: Python program to simulate sequential ATM transactions with balance and daily withdrawal limit validation.

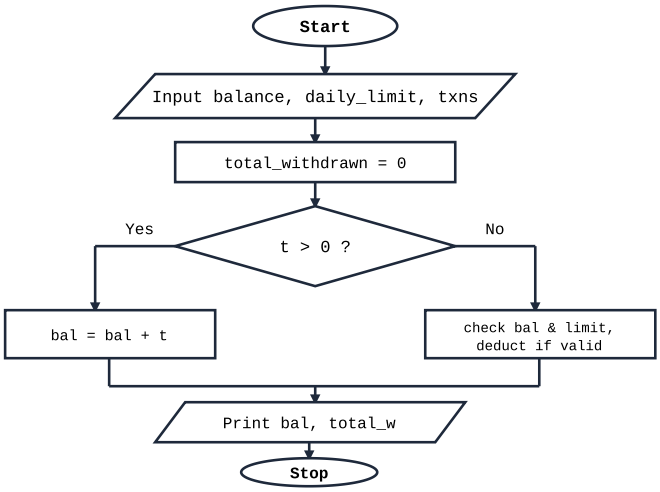
Analyze the problem (Identify the input, process, output):

- **Input:** opening balance, daily withdrawal limit, list of transaction amounts (positive for deposit, negative for withdrawal)
- **Process:** update balance for deposits; for withdrawals check if amount <= balance and total_withdrawn + amt <= daily_limit
- **Output:** transaction status messages, final balance, and total amount withdrawn

Algorithm:

1. Start
2. Input balance and daily_limit
3. Set total_withdrawn = 0
4. For each transaction t in transaction list: if t > 0, balance = balance + t; else amt = -t, check if amt > balance (print insufficient) or total_withdrawn + amt > daily_limit (print limit exceeded), else balance = balance - amt and total_withdrawn = total_withdrawn + amt
5. Print final balance and total_withdrawn
6. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT balance INPUT daily_limit SET total_withdrawn = 0 SET transactions = [1000, -2000, -4000, 500] FOR EACH t IN transactions DO IF t > 0 THEN COMPUTE balance = balance + t PRINT balance ELSE SET amt = -t IF amt > balance THEN PRINT "Insufficient balance" ELSE IF total_withdrawn + amt > daily_limit THEN PRINT "Exceeds daily limit" ELSE COMPUTE balance = balance - amt COMPUTE total_withdrawn = total_withdrawn + amt PRINT balance PRINT balance PRINT total_withdrawn END
```

Python Program:

```
balance = int(input("Enter initial balance: ")) daily_limit = int(input("Enter daily limit: ")) total_withdrawn = 0 transactions = [1000, -2000, -4000, 500] for t in transactions: if t > 0: balance = balance + t print("Deposited:", t, "New Balance:", balance) else: amt = -t if amt > balance: print("Failed: Insufficient balance for", amt) else: if total_withdrawn + amt > daily_limit: print("Failed: Exceeds daily limit for", amt) else: balance = balance - amt total_withdrawn = total_withdrawn + amt print("Withdrew:", amt, "New Balance:", balance) print("Final Balance:", balance) print("Total Withdrawn:", total_withdrawn)
```

Test Cases:

- **Case-1:**
Input: balance=5000, limit=10000, txns=[1000, -2000]
Expected: Final Balance: 4000, Total Withdrawn: 2000
Actual: Final Balance: 4000, Total Withdrawn: 2000

- **Case-2:**
Input: balance=2000, limit=5000, txns=[-3000]
Expected: Insufficient balance, Final Balance: 2000
Actual: Insufficient balance, Final Balance: 2000

LAB Experiment - 4

Problem Statement: Write a python program for Advanced Numeric Pattern Generator to produce a hollow diamond numerical pattern based on user-specified dimensions using nested loops.

Aim: Python program to generate a symmetric hollow diamond number pattern using nested loops.

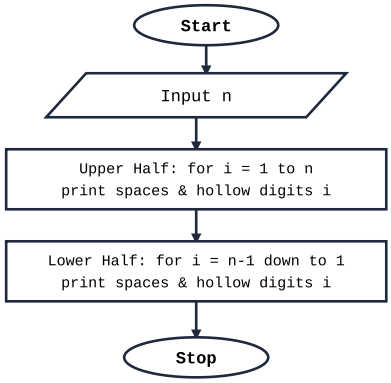
Analyze the problem (Identify the input, process, output):

- **Input:** integer n (number of rows for half pattern)
- **Process:** use outer loop for rows, inner loop for leading spaces, and second inner loop to print numbers only at boundary positions (1 and 2*i-1)
- **Output:** hollow diamond shape printed with row index numbers

Algorithm:

1. Start
2. Input n
3. For i from 1 to n: print (n - i) spaces; for j from 1 to (2*i - 1): print i if j == 1 or j == 2*i-1 else space; print newline
4. For i from n-1 down to 1: print (n - i) spaces; for j from 1 to (2*i - 1): print i if j == 1 or j == 2*i-1 else space; print newline
5. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT n FOR i FROM 1 TO n DO FOR space FROM 1 TO (n - i) DO PRINT " " WITHOUT NEWLINE FOR j FROM 1 TO (2 * i - 1) DO IF j == 1 OR j == (2 * i - 1) THEN PRINT i WITHOUT NEWLINE ELSE PRINT " " WITHOUT NEWLINE PRINT NEWLINE FOR i FROM (n - 1) DOWNTO 1 DO FOR space FROM 1 TO (n - i) DO PRINT " " WITHOUT NEWLINE FOR j FROM 1 TO (2 * i - 1) DO IF j == 1 OR j == (2 * i - 1) THEN PRINT i WITHOUT NEWLINE ELSE PRINT " " WITHOUT NEWLINE PRINT NEWLINE END
```

Python Program:

```
n = int(input("Enter number of rows: ")) for i in range(1, n + 1): for space in range(n - i): print(" ", end="") for j in range(1, 2 * i): if j == 1 or j == (2 * i - 1): print(i, end="") else: print(" ", end="") print() for i in range(n - 1, 0, -1): for space in range(n - i): print(" ", end="") for j in range(1, 2 * i): if j == 1 or j == (2 * i - 1): print(i, end="") else: print(" ", end="") print()
```

Test Cases:

- **Case-1:**
  Input: n=3
  Expected: Hollow diamond with numbers 1, 2, 3
  Actual: Hollow diamond printed correctly

- **Case-2:**
  Input: n=4
  Expected: Symmetric 7-line hollow numeric pattern
  Actual: Symmetric 7-line hollow numeric pattern

LAB Experiment - 5

Problem Statement: Write a python program for Password Policy Validation Engine to evaluate sample password strings against minimum length, character-category (upper, lower, digit), and report validation status.

Aim: Python program to check whether a password string satisfies complexity criteria using basic string methods.

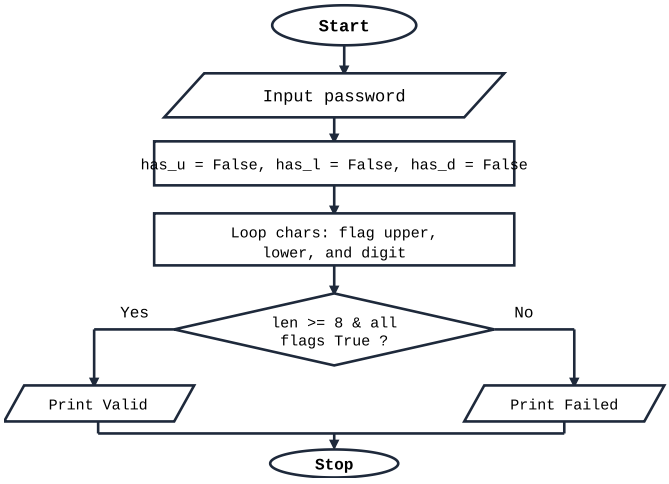
Analyze the problem (Identify the input, process, output):

- **Input:** password string
- **Process:** check length >= 8; iterate through string to check for uppercase, lowercase, and digit characters using boolean flags
- **Output:** validation failure reasons or confirmation message

Algorithm:

1. Start
2. Input password
3. Set has_upper = False, has_lower = False, has_digit = False
4. If length of password < 8: print "Failed: Must be at least 8 chars"
5. For each char ch in password: if ch.isupper() set has_upper=True; if ch.islower() set has_lower=True; if ch.isdigit() set has_digit=True
6. If all conditions met print "Password is valid", else print error
7. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT password SET has_upper = False SET has_lower = False SET has_digit = False IF LENGTH(password) < 8 THEN PRINT "Failed: Password must be at least 8 characters long" FOR EACH ch IN password DO IF ch IS UPPERCASE THEN SET has_upper = True ELSE IF ch IS LOWERCASE THEN SET has_lower = True ELSE IF ch IS DIGIT THEN SET has_digit = True IF LENGTH(password) >= 8 AND has_upper AND has_lower AND has_digit THEN PRINT "Password is valid" ELSE PRINT "Password is invalid" END
```

Python Program:

```
password = input("Enter password: ") has_upper = False has_lower = False has_digit = False if len(password) < 8: print("Failed: Password must be at least 8 characters long") for ch in password: if ch.isupper(): has_upper = True elif ch.islower(): has_lower = True elif ch.isdigit(): has_digit = True if not has_upper: print("Failed: Missing uppercase letter") if not has_lower: print("Failed: Missing lowercase letter") if not has_digit: print("Failed: Missing numeric digit") if len(password) >= 8 and has_upper and has_lower and has_digit: print("Password is valid")
```

Test Cases:

- **Case-1:**
Input: "Pass1234"
Expected: Password is valid
Actual: Password is valid

- **Case-2:**
Input: "pass"
Expected: Failed: at least 8 chars, missing upper, missing digit
Actual: Failed: at least 8 chars, missing upper, missing digit

LAB Experiment - 6

Problem Statement: Write a python program for String-Based Examination Answer Evaluation that compares student answer strings against an answer key, counts correct, incorrect, and unanswered responses, and applies negative marking.

Aim: Python program to grade exam responses against an answer key using string indexing and compute net score.

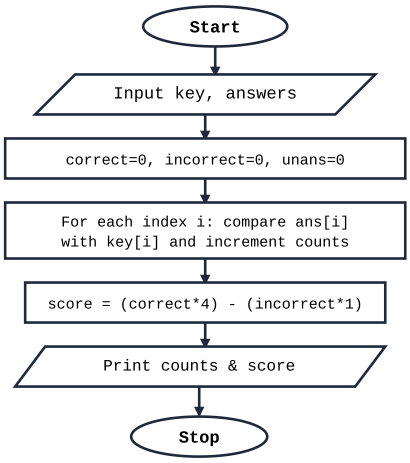
Analyze the problem (Identify the input, process, output):

- **Input:** answer key string and student answer string (using space ' ' for unanswered)
- **Process:** loop over string index; compare characters: correct gives +4, incorrect gives -1, space gives 0
- **Output:** counts of correct, incorrect, unanswered questions and total score

Algorithm:

1. Start
2. Input key and answers strings
3. Set correct = 0, incorrect = 0, unanswered = 0
4. For i from 0 to length(key) - 1: if answers[i] == ' ', unanswered += 1; else if answers[i] == key[i], correct += 1; else incorrect += 1
5. Compute score = (correct * 4) - (incorrect * 1)
6. Print correct, incorrect, unanswered, and score
7. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT key INPUT answers SET correct = 0 SET incorrect = 0 SET unanswered = 0 FOR i FROM 0 TO LENGTH(key) - 1 DO IF answers[i] == " " THEN COMPUTE unanswered = unanswered + 1 ELSE IF answers[i] == key[i] THEN COMPUTE correct = correct + 1 ELSE COMPUTE incorrect = incorrect + 1 COMPUTE score = (correct * 4) - (incorrect * 1) PRINT correct PRINT incorrect PRINT unanswered PRINT score END
```

Python Program:

```
key = input("Enter answer key: ") answers = input("Enter student answers: ") correct = 0 incorrect = 0 unanswered = 0 for i in range(len(key)): if answers[i] == " ": unanswered = unanswered + 1 elif answers[i] == key[i]: correct = correct + 1 else: incorrect = incorrect + 1 score = (correct * 4) - (incorrect * 1) print("Correct:", correct) print("Incorrect:", incorrect) print("Unanswered:", unanswered) print("Final Score:", score)
```

Test Cases:

- **Case-1:**
Input: key="ABCDABCD", ans="AB CDABC"
Expected: Correct: 6, Incorrect: 1, Unanswered: 1, Score: 23
Actual: Correct: 6, Incorrect: 1, Unanswered: 1, Score: 23

- **Case-2:**
Input: key="AAAA", ans="AAAA"
Expected: Correct: 4, Incorrect: 0, Unanswered: 0, Score: 16
Actual: Correct: 4, Incorrect: 0, Unanswered: 0, Score: 16

LAB Experiment - 7

Problem Statement: Write a python program for Text Compression Using Character Runs that encodes a string by grouping consecutive repeated characters and compares original and encoded lengths.

Aim: Python program to perform run-length encoding on strings and determine compression benefit.

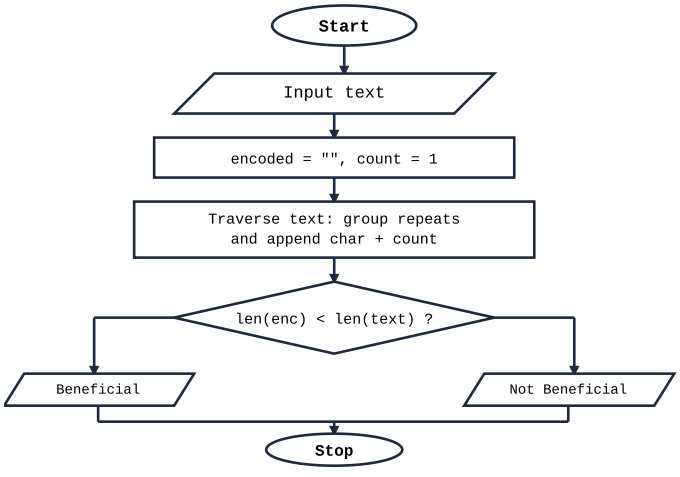
Analyze the problem (Identify the input, process, output):

- **Input:** text string
- **Process:** count repeated consecutive characters; append char and count to compressed string; compare length
- **Output:** compressed string and benefit evaluation message

Algorithm:

1. Start
2. Input text
3. Set encoded = "", count = 1
4. For i from 0 to len(text) - 1: if i + 1 < len(text) and text[i] == text[i+1], count = count + 1; else encoded = encoded + text[i] + str(count), count = 1
5. If len(encoded) < len(text) print "Beneficial", else print "Not beneficial"
6. Stop

Flowchart:



Pseudocode:

```
BEGIN INPUT text SET encoded = "" SET count = 1 FOR i FROM 0 TO LENGTH(text) - 1 DO IF i + 1 < LENGTH(text) AND text[i] == text[i + 1] THEN COMPUTE count = count + 1 ELSE COMPUTE encoded = encoded + text[i] + STRING(count) SET count = 1 PRINT text PRINT encoded IF LENGTH(encoded) < LENGTH(text) THEN PRINT "Compression is beneficial" ELSE PRINT "Compression is not beneficial" END
```

Python Program:

```
text = input("Enter text: ") encoded = "" count = 1 for i in range(len(text)): if i + 1 < len(text) and text[i] == text[i + 1]: count = count + 1 else: encoded = encoded + text[i] + str(count) count = 1 print("Original Text:", text) print("Encoded Text:", encoded) if len(encoded) < len(text): print("Compression is beneficial") else: print("Compression is not beneficial")
```

Test Cases:

- **Case-1:**
Input: "aaabbccccd"
Expected: Encoded: "a3b2c4d1", Beneficial
Actual: Encoded: "a3b2c4d1", Beneficial

- **Case-2:**
Input: "abcd"
Expected: Encoded: "a1b1c1d1", Not beneficial
Actual: Encoded: "a1b1c1d1", Not beneficial

LAB Experiment - 8

Problem Statement: Write a python program for Student Marks Matrix Analysis that analyses a 2D matrix of student marks across subjects, calculates row-wise and column-wise averages, and finds the top performer.

Aim: Python program to process nested lists of student scores to calculate student and subject averages.

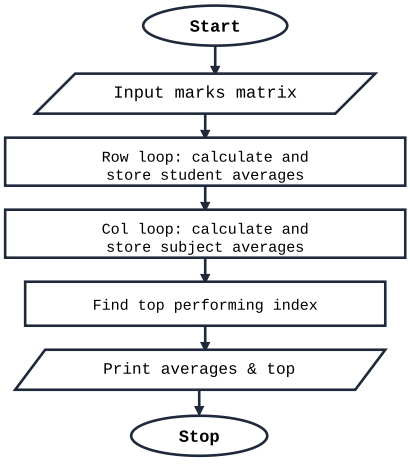
Analyze the problem (Identify the input, process, output):

- **Input:** nested list representing marks matrix (rows: students, columns: subjects)
- **Process:** compute row totals for student averages; column totals for subject averages; search max student average
- **Output:** list of student averages, subject averages, and index of top student

Algorithm:

1. Start
2. Initialize matrix with student marks
3. For each row in matrix: compute total and append total / num_subjects to student_averages
4. For each column j: compute column sum across rows and append col_sum / num_students to subject_averages
5. Find index with highest student average
6. Print student_averages, subject_averages, top_student
7. Stop

Flowchart:



Pseudocode:

```
BEGIN SET matrix = [[75, 80, 85], [60, 55, 58], [90, 92, 88]] SET num_students = LENGTH(matrix) SET num_subjects = LENGTH(matrix[0]) SET student_averages = [] FOR EACH row IN matrix DO SET total = 0 FOR EACH mark IN row DO COMPUTE total = total + mark APPEND (total / num_subjects) TO student_averages SET subject_averages = [] FOR j FROM 0 TO num_subjects - 1 DO SET col_total = 0 FOR i FROM 0 TO num_students - 1 DO COMPUTE col_total = col_total + matrix[i][j] APPEND (col_total / num_students) TO subject_averages SET top_student = 0 SET highest_avg = student_averages[0] FOR i FROM 0 TO LENGTH(student_averages) - 1 DO IF student_averages[i] > highest_avg THEN SET highest_avg = student_averages[i] SET top_student = i PRINT student_averages PRINT subject_averages PRINT top_student END
```

Python Program:

```
matrix = [ [75, 80, 85], [60, 55, 58], [90, 92, 88], [40, 50, 45] ] num_students = len(matrix) num_subjects = len(matrix[0]) student_averages = [] for row in matrix: total = 0 for mark in row: total = total + mark student_averages.append(total / num_subjects) subject_averages = [] for j in range(num_subjects): col_total = 0 for i in range(num_students): col_total = col_total + matrix[i][j] subject_averages.append(col_total / num_students) print("Student Averages:", student_averages) print("Subject Averages:", subject_averages) top_student = 0 highest_avg = student_averages[0] for i in range(len(student_averages)): if student_averages[i] > highest_avg: highest_avg = student_averages[i] top_student = i print("Top Student Index:", top_student)
```

Test Cases:

- **Case-1:**
  Input: 4x3 matrix as defined in program
  Expected: Top Student Index: 2 (Avg: 90.0)
  Actual: Top Student Index: 2 (Avg: 90.0)
- **Case-2:**
  Input: matrix = [[50, 50], [80, 80]]
  Expected: Student Avgs: [50.0, 80.0], Top: 1
  Actual: Student Avgs: [50.0, 80.0], Top: 1

LAB Experiment - 9

Problem Statement: Write a python program for Tuple-Based Product Catalogue Audit that stores product identifiers, prices, and stock in tuples, detects invalid records, calculates inventory values, and identifies items needing reorder.

Aim: Python program to unpack catalogue tuples, validate entries, and audit inventory stock.

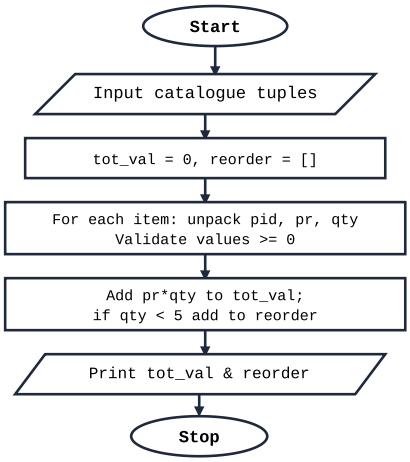
Analyze the problem (Identify the input, process, output):

- **Input:** tuple of product records (pid, price, qty)
- **Process:** unpack each item; validate price >= 0 and qty >= 0; compute value = price * qty; flag items with qty < 5
- **Output:** total inventory value and list of products requiring replenishment

Algorithm:

1. Start
2. Initialize catalogue tuple of tuples
3. Set total_inventory_value = 0, reorder_list = []
4. For each item in catalogue: unpack (pid, price, qty); if price < 0 or qty < 0 print invalid; else add price*qty to total, and if qty < 5 append pid to reorder_list
5. Print total_inventory_value and reorder_list
6. Stop

Flowchart:



Pseudocode:

```
BEGIN SET catalogue = (("P101", 150, 10), ("P102", -20, 5), ("P103", 45, 0), ("P104", 80, 2)) SET total_inventory_value = 0 SET reorder_list = [] FOR EACH item IN catalogue DO UNPACK item INTO pid, price, qty IF price < 0 OR qty < 0 THEN PRINT "Invalid item record found:" PRINT pid ELSE COMPUTE total_inventory_value = total_inventory_value + (price * qty) IF qty < 5 THEN APPEND pid TO reorder_list PRINT total_inventory_value PRINT reorder_list END
```

Python Program:

```
catalogue = ( ("P101", 150, 10), ("P102", -20, 5), ("P103", 45, 0), ("P104", 80, 2) ) total_inventory_value = 0 reorder_list = [] for item in catalogue: pid, price, qty = item if price < 0 or qty < 0: print("Invalid item record found:", pid) else: total_inventory_value = total_inventory_value + (price * qty) if qty < 5: reorder_list.append(pid) print("Total Inventory Value:", total_inventory_value) print("Products needing reorder:", reorder_list)
```

Test Cases:

- **Case-1:**
Input: Defined 4-tuple catalogue
Expected: Invalid: P102, Value: 1660, Reorder: ['P103', 'P104']
Actual: Invalid: P102, Value: 1660, Reorder: ['P103', 'P104']

- **Case-2:**
Input: (("A1", 100, 10), ("A2", 200, 1))
Expected: Value: 1200, Reorder: ['A2']
Actual: Value: 1200, Reorder: ['A2']

LAB Experiment - 10

Problem Statement: Write a python program for Bus Passenger Occupancy Simulation to process boarding and alighting counts across multiple stops, calculate running passenger count, and flag capacity violations.

Aim: Python program to track bus occupancy across stops and detect capacity overflow.

Analyze the problem (Identify the input, process, output):

- **Input:** bus capacity and list of stops with tuples (boarding, alighting)
- **Process:** for each stop update current_passengers = current_passengers - alight + board; if passengers > capacity, flag violation
- **Output:** passenger count at each stop and total violation count

Algorithm:

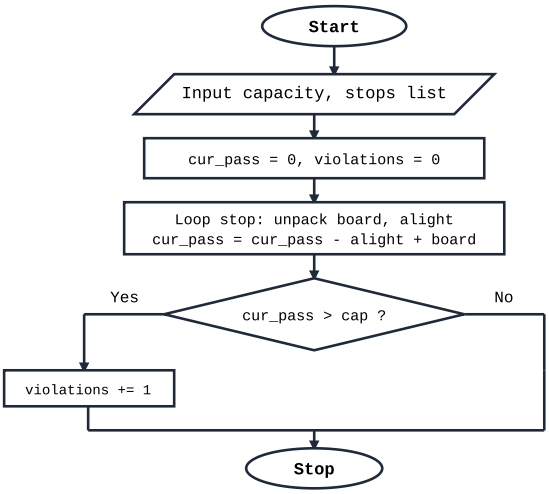
1. Start
2. Input capacity and list of stop tuples (board, alight)
3. Set current_passengers = 0, violations = 0, stop_num = 1

4. For each stop in stops: unpack (board, alight); current_passengers = current_passengers - alight + board; if current_passengers > capacity increment violations; stop_num = stop_num + 1

5. Print total violations

6. Stop

Flowchart:



Pseudocode:

```
BEGIN SET capacity = 40 SET current_passengers = 0 SET violations = 0 SET stops = [(15, 0), (10, 5), (25, 2), (5, 10)] SET stop_num = 1 FOR EACH stop IN stops DO UNPACK stop INTO board, alight COMPUTE current_passengers = current_passengers - alight + board PRINT stop_num PRINT current_passengers IF current_passengers > capacity THEN PRINT "Capacity violation at stop" PRINT stop_num COMPUTE violations = violations + 1 COMPUTE stop_num = stop_num + 1 PRINT violations END
```

Python Program:

```
capacity = int(input("Enter bus capacity: ")) current_passengers = 0 violations = 0 stops = [ (15, 0), (10, 5), (25, 2), (5, 10) ] stop_num = 1 for stop in stops: board, alight = stop current_passengers = current_passengers - alight + board print("Stop", stop_num, "- Passengers on board:", current_passengers) if current_passengers > capacity: print("Capacity violation at stop", stop_num) violations = violations + 1 stop_num = stop_num + 1 print("Total Capacity Violations:", violations)
```

Test Cases:

- **Case-1:**

Input: capacity=40, stops=[(15,0),(10,5),(25,2),(5,10)]
Expected: Violation at stop 3 (passengers=48), Total: 1
Actual: Violation at stop 3 (passengers=48), Total: 1

- **Case-2:**

Input: capacity=50, stops=[(20,0), (10,5)]
Expected: Stop 1: 20, Stop 2: 25, Violations: 0
Actual: Stop 1: 20, Stop 2: 25, Violations: 0